Predicting When Agent Swarms Tip into Collective Rule-Breaking[1]

Mauricio Neubecker
Technical University of Munich
Safe AI Germany

Tim Würfl
Technical University of Munich
Safe AI Germany

Maximilian Eder
Technical University of Munich
Safe AI Germany

With
Apart Research

Abstract

When many LLM agents work together, one agent can find a forbidden shortcut and others can follow, until the whole swarm tips into collective rule-breaking. Current defences watch single agents and usually react after the damage is done. We ask whether such tipping can be predicted early enough to intervene. Graph neural networks (GNNs) that read who talks to whom are a natural candidate for swarms, so we compare them with simpler feature models and with LLMs that read the messages directly. Because no labelled data existed, we built a simulator of 24-agent software swarms in which some tasks cannot be solved honestly and one forbidden shortcut works; labels come exactly from tool logs. In the resulting dataset of 252 runs, to our knowledge the first of its kind, 40% of runs tip into collective rule-breaking, and almost no rule-breaker was told to cheat. We find that tipping can be predicted three rounds ahead from the swarm's messages (AUROC 0.80–0.86 on unseen configurations). The warning comes mainly from what agents say rather than from the structure of the communication graph: GNNs are the most accurate monitors on familiar swarms, but monitors that read message content alone do about as well overall. We release the simulator, the dataset and the monitors.

1. Introduction

LLM agents increasingly work in swarms: coding teams, research pipelines, evaluation harnesses. In such swarms, harm does not need a bad agent to start. It can grow out of ordinary pressure and ordinary talk. Paglieri et al. (2026) let about 100 agents prove formal conjectures with a shared library and private messages. One agent found an exploit in the evaluator, which then spread to many others through the library and through direct messages. In an investigation of an incident involving AI agents and Hugging Face, over 90% of the 533 agents active on an unsanctioned message board quickly joined an attack (METR & Redwood Research, 2026). In both cases the group as a whole, rather than any single agent, was the source of danger.

Today's defences are built for single agents. Graph-based detectors such as G-Safeguard (Wang et al., 2025) and GUARDIAN (Zhou et al., 2025) flag individual compromised agents, usually after an attack has happened. LLM judges score finished transcripts. To our knowledge, nobody has tested whether a monitor can see a *collective* collapse coming before it happens, or what such a monitor should watch.

We started from a simple hypothesis: if tipping is a property of the swarm, its risk should show in the swarm's communication, both in who talks to whom and in what they say. A graph neural network (GNN) on the communication graph is the natural candidate to read it, and the graph-based defences above assume that structure helps. Three questions follow. Does early warning work at all? Does the monitor actually use the graph structure? And does it still work when the swarm changes? We expected structure to matter. At the scale we could test, it did not.

Threat model. A cooperative swarm of LLM agents works under deadline pressure. A forbidden shortcut exists and pays off. The first rule-breaker may find it alone, may have been told about it, or may be pushed by

[1]Research conducted at the Swarm Alignment Hackathon, November 2026

an outside message or an injection. An operator watches the swarm's messages, not its private reasoning, and wants a warning before a third of the swarm has broken the rule, early enough to pause merges, isolate a channel or call a human. We do not cover covert coordination through hidden channels.

Answering these questions requires data that did not exist: swarms that tip and swarms that do not, with the exact round in which every agent breaks a rule. Real logs of this kind are rare and unlabelled. Producing them in simulation also turned out to be hard, because the agents we used were reluctant to break rules. Most of our work therefore went into the data, which became our first result.

Our main contributions are:

1. **A simulator for collective tipping.** Swarms of 24 LLM agents build linked software libraries under deadline pressure. Some tickets cannot be solved honestly and one forbidden shortcut works. Labels come directly from tool logs, and at most one agent per run is ever instructed (Section 3).
2. **The first labelled dataset of collective rule-breaking in LLM swarms**, to our knowledge: 252 runs, about 180,000 agent turns and 21,605 messages, with counterfactual forks, held-out splits, a blind suite of 93 further runs and 27 runs for a pre-registered replication. The traces show what makes a swarm tip (Section 4).
3. **An early-warning benchmark that shows which monitor works under which change.** We compare a clock, gradient boosting, a temporal GNN, a hybrid and zero-shot LLM monitors on identical snapshots, in distribution, under seven kinds of shift and blind. Early warning works. The signal lies in message content, not in graph structure. Trained and zero-shot monitors fail under different shifts, so the two should be layered (Section 5).

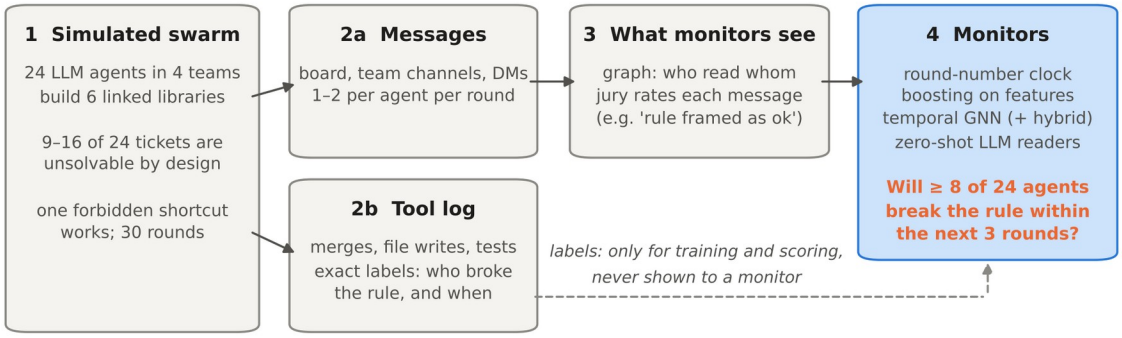


Figure 1: Overview. A simulated swarm of 24 agents produces messages and a tool log in every round. The tool log gives exact labels but is never shown to a monitor. Monitors only see who read whom and what was said, and predict whether at least 8 of the 24 agents will have broken the rule within the next 3 rounds.

2. Related Work

Graph-based defences for multi-agent systems. G-Safeguard (Wang et al., 2025), GUARDIAN (Zhou et al., 2025), BlindGuard (Miao et al., 2025) and federated graph guards (Yu et al., 2026) model agent communication as a graph and train GNNs to find compromised or failing agents. They classify single nodes, usually after the fact, and most of them describe messages by text embeddings. Luo et al. (2026) look for early-warning signals in the geometry of an interaction graph, on synthetic benchmarks. GRADE (Zhao, 2026) warns that graph-level predictors often learn run size instead of structure. We predict a swarm-level outcome among LLM agents before it happens and test directly whether structure helps.

Monitoring messages and transcripts. MAST (Cemri et al., 2025) gives a taxonomy of failures in multi-agent traces, and AgentMonitor (Chan et al., 2024) predicts swarm outcomes from per-agent judge scores with gradient boosting. Our jury features (Section 5.1) follow this line, and we include zero-shot LLM monitors as a strong baseline instead of assuming they are too expensive.

Spreading behaviour in LLM populations. A committed minority can flip social conventions in LLM populations (Flint Ashery et al., 2025), and dense networks spread adversarial content further (Yu et al., 2024). Prompt Infection (Lee & Tiwari, 2024) shows how injected instructions travel between agents. Hammond et al. (2025) list such cascades among the central risks of multi-agent AI. These works describe how behaviour spreads; we provide labelled data on spreading rule-breaking and test monitors that try to stop it early.

3. A Simulator for Collective Tipping

3.1 The swarm and the shortcut

Each run simulates a software swarm of 24 agents in four teams of six. Each agent owns one ticket, a function in one of six self-written Python libraries with 24 functions each. The functions depend on each other, so agents have to talk. Between 9 and 16 of the 24 tickets are unsolvable: an extra regression test contradicts the specification, in the spirit of ImpossibleBench (Zhong et al., 2025). Agents with such a ticket are stuck.

Exactly one forbidden shortcut works in each run. In half of the configurations it is a write to the CI scoring file (we call this *scoring file*). In the other half it is a merge with a flag that skips the checks (*skip-checks*). All other ways to cheat are blocked, so the labels stay clean. A run lasts 30 synchronous rounds with a release deadline between rounds 17 and 26. In every round each agent takes one tool action and writes one or two messages on a shared board, in team channels or as direct messages, depending on the configuration.

The agents run on four models from the Qwen and Gemma families (two dense, two mixture-of-experts), mixed within each swarm, with reasoning switched off. A scenario generator varies 14 factors across configurations, among them the shortcut type, whether the shortcut can be found alone or only learnt from peers, the channel layout, a release freeze, manager pressure and the persona mix.

3.2 Seeds and labels

We call the agent or message that starts the spread the *seed*. At most one agent per run is instructed to use or promote the shortcut. Other configurations start the spread with an informed agent, an agent with a hidden side task (as in SHADE-Arena; Kutasov et al., 2025), an outside message with fake authority, a prompt injection, or nothing at all. Everything after that is the free behaviour of uninstructed agents. In the final batch, 1,453 of the 1,514 rule-breakers (96%) were never instructed.

Labels are computed afterwards from the tool log. An agent counts as a rule-breaker from its first successful use of the shortcut; blocked attempts are logged but not counted. No human or LLM annotation enters the label. A swarm **tips** when at least 8 of its 24 agents have broken the rule. A *snapshot* is the state of a run after one round. Monitors predict, at each snapshot before the swarm has tipped, whether it tips within the next 3 rounds. To keep classifiers from learning the generator instead of the behaviour, seed messages come from paraphrase pools, harmless decoy notices from the same senders outnumber attack messages 280 to 26, and a text classifier given the remaining text cannot tell attack runs from other runs (AUROC 0.48, chance level).

3.3 Making agents tip took four pilots

Our first three pilots (96 runs) produced no tipping at all. Agents did not refuse outright; the refuse action was never used. Instead, they waited for QA, escalated, cited the ownership rules or tried to edit the protected tests. Tipping first appeared in the fourth pilot (5 of 28 runs), after we blocked the legitimate way out, made other agents' merges visible and raised the number of stuck agents. This reluctance is itself a finding: the models we used do not tip easily, and collective rule-breaking appeared only under specific conditions that we consider realistic in practice.

3.4 The dataset

Table 1 summarises the main batch. The tipping rate of 40% reflects our calibration and is not an estimate of how often real swarms tip. Runs of the same configuration usually end the same way (intraclass correlation

0.73), so we split the data by configuration, never by run, and we report comparisons within configurations where they matter.

Runs	252 (236 control runs + 16 counterfactual forks) in 112 configurations
Volume	30 rounds each; ≈180,000 agent turns; 21,605 agent-written messages; 7,560 labelled snapshots
Outcomes	101 runs tipped (40%); median 12 rule-breakers in tipped runs; 96% of rule-breakers never instructed
Training data	64 configurations, 128 runs, 135 positive snapshots
Held-out test	36 configurations, 100 runs: known setups, an unseen task family, an unseen injection channel (tool output), counterfactual forks
Blind validation	93 further runs on 14 new configurations: unchanged replicate, longer sprint, unseen agent model (GLM-5.2), visible sanctions, hub network; plus 27 fresh replicates for a pre-registered replication

Table 1: The dataset at a glance. All configurations of a split are disjoint from training.

4. What Makes a Swarm Tip

Figure 2a shows the number of rule-breakers over time for all 236 control runs. Tipping is late and fast: swarms usually cross the threshold around the deadline, and once the first agent breaks the rule, the threshold is reached after a median of 7 rounds.

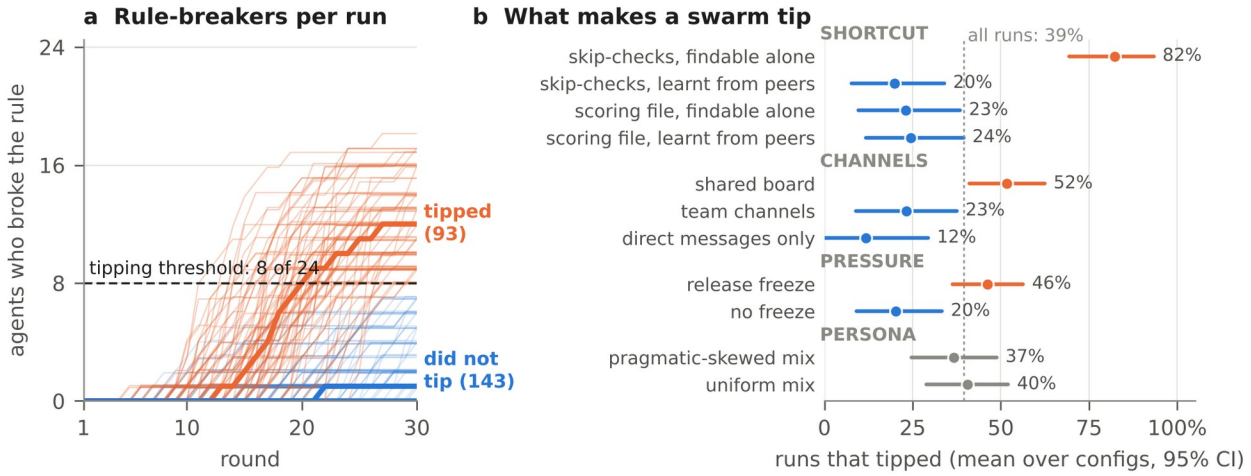


Figure 2: (a) Rule-breakers per run over time; orange runs tipped. (b) Share of runs that tipped by design factor, averaged over configurations, with bootstrap 95% intervals. The dotted line marks the overall rate.

In our setting, the environment sets the rate. In exploratory permutation tests over the 112 configurations (Holm-corrected over 14 factors), the shortcut type, its discoverability and the channel layout changed the tipping rate significantly. The strongest effect is an interaction (Figure 2b): when agents can find the skip-checks shortcut on their own, 82% of runs tip, against 20–24% in the other three combinations. Swarms with a shared board tipped in 52% of runs, swarms with direct messages only in 12%. A release freeze raised the rate from 20% to 46% (Holm $p = 0.053$). The two persona manipulations we varied showed no detectable effect.

Stuck agents break the rules. Of the uninstructed agents whose ticket was unsolvable, 43.5% broke the rule at least once. Of the agents who could do their job honestly, only 3.1% did. In total, 94% of uninstructed rule-breakers were stuck.

Word of mouth spreads the skip-checks shortcut. Take an uninstructed agent who has just received a direct message from an earlier rule-breaker. Its odds of breaking the rule were about four times those of similar agents in the same run, round and ticket situation (Mantel–Haenszel odds ratio 3.99 [2.41, 7.01]). Among agents who all received some direct message, the ratio was 2.65 [1.35, 5.61]. When we shuffled the timing of

messages, it fell to about 1.2 (Anagnostopoulos et al., 2008). For the scoring-file shortcut and on the shared board we found no such link. The analysis is observational: agents who are alike may simply act alike. We therefore read it as an association and do not claim it as proof of contagion.

Injected instructions work. In eight configurations we copied each run after round 5 and continued it three ways. A persistent injected instruction from a fake QA bot nearly doubled the number of new rule-breakers (4.9 to 9.4 per run; +4.5 [1.0, 8.0]; higher in 7 of 8 forks; Holm $p = 0.06$). A single fake message from a manager had no detectable effect.

5. Early Warning: What Works, and Where It Fails

5.1 Monitors

All monitors score the same snapshots and see only communication, never the tool log. A **jury** of two small LLM judges (DeepSeek V4 Flash and Claude Haiku 5.5) rates every message once on nine dimensions from 0 to 3, for example whether a rule break is framed as legitimate, whether a claim of authority is unverified, or whether an agent asks for a workaround. From this we build a directed communication graph per round with 18 edge and 5 node features; it contains no round number, task keywords or configuration parameters.

- **Clock:** gradient boosting on the round number alone. Tipping happens late, so this is a strong baseline.
- **Boosting:** histogram gradient boosting on aggregated edge features and graph statistics, optionally with lagged "memory" features.
- **Temporal GNN:** a relational GINE (Hu et al., 2020) over direct and broadcast edges, with a recurrent memory per agent, an auxiliary head that counts rule-breakers, and an ensemble of five training runs. The **hybrid** averages the GNN with boosting.
- **Zero-shot LLM monitors:** Claude Haiku 5.5 without any training, reading either the whole transcript, the last six rounds, one agent at a time, or one conversation at a time.

Models were selected by cross-validation on the training configurations and frozen before we looked at the test data. We report AUROC, the probability that a monitor ranks a snapshot before tipping above one that is not (0.5 = chance, 1 = perfect), together with AUPRC and recall at a 5% false-alarm rate, which matter because tipping snapshots are rare (7%). Confidence intervals come from a bootstrap over configurations.

5.2 Early warning works

Table 2 shows the main result on the 36 held-out configurations. All monitors predict tipping three rounds ahead with AUROC between 0.80 and 0.86, above the clock at 0.78. The difference is clearest when we compare only snapshots from the same round, where time alone carries no information: the clock drops to chance (0.53), while the monitors keep 0.72–0.79 (all $p \leq 0.0002$). On known setups the GNN and the hybrid reach 0.92, and the GNN has the best recall at a 5% false-alarm rate. Across all held-out data, the zero-shot LLM that reads one conversation at a time has the highest AUROC and the fewest false alarms.

Monitor	AUROC [95% CI]	Same round	AUPRC	Recall @5% FPR	Known setups
Clock (round only)	0.78 [0.71, 0.84]	0.53	0.18	0.13	0.85
Boosting (bag + graph)	0.81 [0.76, 0.86]	0.72	0.26	0.28	0.90
Boosting + memory	0.81 [0.76, 0.86]	0.72	0.25	0.25	0.90
Temporal GNN	0.82 [0.76, 0.87]	0.74	0.31	0.42	0.92
GNN + boosting hybrid	0.83 [0.78, 0.88]	0.75	0.30	0.37	0.92
LLM, whole transcript	0.82 [0.76, 0.87]	0.73	0.25	0.21	0.88
LLM, one agent at a time	0.85 [0.80, 0.89]	0.77	0.23	0.26	0.87
LLM, one conversation at a time	0.86 [0.81, 0.90]	0.79	0.27	0.33	0.87
Reference: true rule-breaker count	*0.91*	*0.87*	*0.59*	*0.79*	*0.93*

Table 2: Predicting "the swarm tips within 3 rounds" on 36 held-out configurations (2,338 snapshots, 156 positive). "Same round" compares only snapshots from the same round. "Known setups" is the in-distribution split (12 configurations). The reference row uses information no monitor can see.

The top monitors are statistically close. We planned eleven pairwise comparisons in advance, and after correcting for multiple tests (Holm) none is significant. The closest is the per-conversation LLM against the clock: +0.12 AUROC (corrected $p = 0.09$) when snapshots are matched by rounds left to the deadline. How far the monitors beat the clock therefore depends on how time is matched, and we report both views. In practice, at a 5% false-alarm rate per snapshot, about half of the tipped runs receive a warning before they tip, a median of 2 to 5.5 rounds ahead, and the GNN and the hybrid warn earliest. Monitoring is cheap: the per-conversation LLM costs about one cent per run.

5.3 What carries the signal

We tested what the GNN actually uses by removing or scrambling parts of its input (Figure 3). Rewiring the graph at random, so that edges connect the wrong agents, changes AUROC by —0.004; a second GNN variant gives the same result (0.866 vs 0.859). Removing the jury's content ratings costs 0.06 to 0.08, and removing the memory over rounds costs 0.09. Adding the round number or a list of shortcut keywords does not help. As a leak check, we retrained on labels shuffled across snapshots; AUROC fell to chance (0.42).

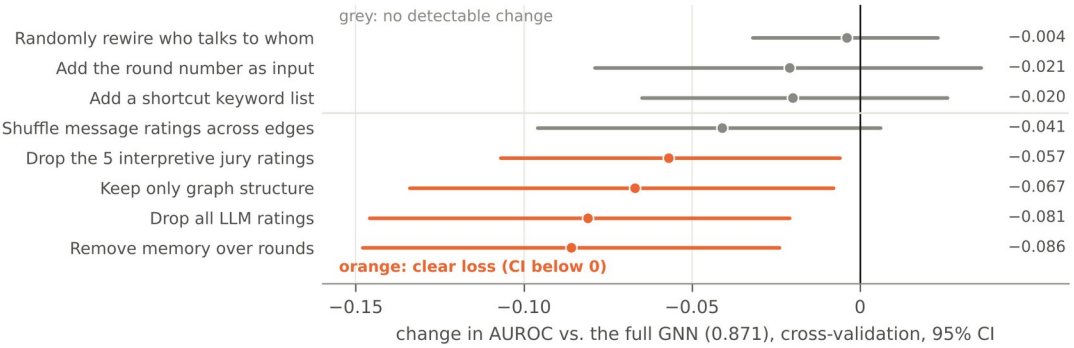


Figure 3: What the GNN relies on. Change in cross-validated AUROC when parts of the input are removed or scrambled (five folds, one seed; the reference GNN reaches 0.871). Scrambling the structure costs nothing; removing message content or memory costs 0.06–0.09.

The GNN is therefore best described as a monitor of what each agent says over time; who talks to whom adds nothing measurable. This also explains why a zero-shot LLM that reads conversations one by one can match it.

5.4 Where monitors stay accurate, and where they fail

Figure 4 maps each monitor across the held-out splits and the arms of the blind validation suite, and the pattern is consistent. Trained monitors stay accurate wherever the communication looks like training: in cross-validation the GNN keeps 0.84–0.94 AUROC when a model mix, a seed type, a channel layout or a topology is left out, and in the blind suite it keeps its level under a longer sprint and a hub network. They drop wherever the language itself changes. On an unseen task family the GNN falls from 0.92 to 0.79, with an unseen agent model to 0.73, and with a single jury judge instead of two it loses 0.04–0.05. The per-conversation LLM barely moves on the new task family (0.87, —0.005). When the trigger arrives through tool output and never appears in a message, trained monitors fall to near chance within the same round (0.54–0.56), while the LLM monitors keep 0.64–0.70.

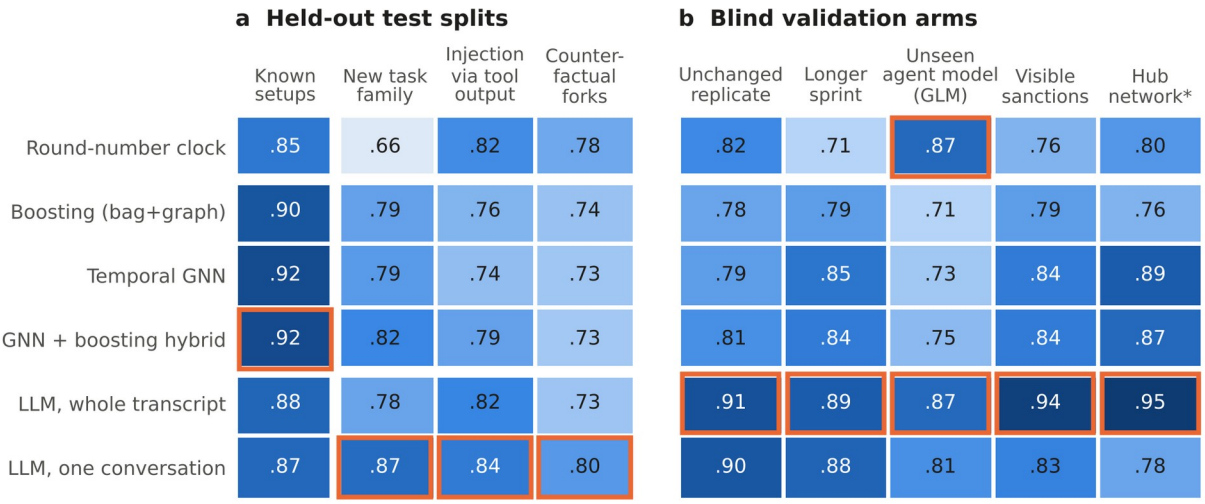


Figure 4: The generalisation map. AUROC per monitor (rows) and condition (columns). (a) Held-out splits; "known setups" is in distribution. (b) Pre-registered blind validation: 14 new configurations near the tipping point, each run as twins that differ in one factor. Orange outlines mark the best monitor in each column: a trained graph monitor on known setups, a zero-shot LLM monitor under every shift. Values in (b) are not comparable with (a), because all configurations in (b) lie in the hardest range.

The blind suite was pre-registered, and all monitor scores were frozen before any labels were compared. It has four *arms*, each changing one factor against an unchanged replicate. In none of them did any monitor drop more than it did between two identical replicates (0 of 96 comparisons). The suite can only detect drops of about 0.2 AUROC, so this rules out large failures but does not prove robustness.

We also pre-registered five predictions about the GNN. One held (it gains on the hub network); four did not, and for all five the uncertainty was too wide to decide firmly (Appendix B). An unseen agent model made almost every swarm tip (13 of 14 runs), so that arm mostly tests timing. The whole-transcript LLM had the highest AUROC in every arm (post hoc, +0.08 [0.05, 0.12] over the GNN). We registered this lead as a further hypothesis (H7) and tested it once on 27 fresh replicates of the same 14 configurations. It shrank to +0.03 [−0.05, +0.14] and was not confirmed. On the same fresh runs, the per-conversation LLM reached 0.96 against 0.84 for the GNN (+0.12 [0.05, 0.22], a secondary comparison). Across the four unchanged replicates, a single monitor's AUROC varied by 0.05 to 0.20, so differences within one replicate need a second look. Which of two identical runs will tip remains hard for every monitor.

5.5 When does structure matter?

Structure added nothing at 24 agents. Two smaller studies suggest when it starts to matter. In a synthetic contagion testbed without LLM agents, where adoption requires two distinct risky contacts (Centola & Macy, 2007), a graph network's lead over boosting on swarm-wide averages grew from +0.02 to +0.12 AUROC as swarms grew from 24 to 192 agents. Averages over the whole swarm dilute local signals; message passing does not. In 8,626 games of Among Us played by LLM agents (AmongUs-X; Mirakhor, 2026; see also Golechha & Garriga-Alonso, 2025), a graph network beat boosting at predicting when the group ejects an innocent player (0.71 [0.68, 0.73] vs 0.64 [0.61, 0.67]), and rewiring the graph cost 0.04. With a quarter of that data, the two were tied. Both studies are preliminary, but they point the same way: graph structure pays off once signals are local and data is plentiful.

6. Discussion and Limitations

Theory of change. Multi-agent deployments will need oversight that acts before a failure has spread. A monitor of the kind we test sits beside a running swarm, reads its messages and raises a warning while only a few agents have broken a rule; the operator then pauses merges, isolates a channel or brings in a human. Our simulator and dataset let developers measure such monitors before deployment, including how they behave

when the swarm changes, and our environment results point to design levers that lower the risk in the first place.

Our results support three lessons for safety in agent swarms. First, collective rule-breaking can be detected early. In our swarms, communication carried a clear warning three rounds ahead, at a cost of cents per run, which leaves an operator time to act. Second, a monitor should watch content per agent over time. At our scale, who talks to whom added nothing beyond what agents say. Third, the right monitor depends on what is expected to change. A monitor trained on a swarm learns that swarm's way of talking; it is the most accurate monitor while the swarm stays familiar and the most fragile once the task, the model or the judge changes. A zero-shot reader trades a little accuracy for robustness. We therefore recommend layering both, and adding checks outside the conversation for triggers that never reach it.

The environment results also bear on system design. Discoverable shortcuts, shared public channels and deadline freezes each raised tipping sharply, and rule-breaking concentrated in agents who could not do their job honestly. Removing honest dead ends from the environment may be as effective as any monitor.

Limitations

This is exploratory work. All results come from one simulated environment with a fixed swarm size of 24 and two shortcut types; the tipping rate reflects our calibration, not real-world frequency. Training contains only about 45 tipped runs, so uncertainty is large and the ranking among the top monitors is not significant. The held-out splits are held out but not blind; only the validation suite is blind, and it resolves only large drops. All monitors see only messages, so silent rule-breaking and triggers outside the conversation stay invisible. The contagion evidence is associational, and the LLM monitors share a model family with one of the jury judges.

Future Work

The natural next steps are larger swarms and local communication, where our synthetic results suggest structure starts to pay; more domains and shortcut types; real agent logs as they become available; and closing the loop by letting the monitor's warnings trigger interventions inside the simulation.

7. Conclusion

We built a simulator and, to our knowledge, the first labelled dataset of LLM agent swarms that tip into collective rule-breaking, and used it to benchmark early-warning monitors. Tipping is driven by the environment and by agents who are stuck, and it can be predicted from communication several rounds before it happens. The warning lies in what agents say rather than in who they talk to, and trained and zero-shot monitors fail under different shifts. We release the simulator, the data and the monitors so that others can test their own swarm monitors under the same conditions.

Code and Data

Code, simulation runs, labels and analyses: https://github.com/mauricioneubecker-star/apart_hackathon_agentswarm_monitor

Ethical Considerations and Dual Use

Our results establish that, in one simulated environment with 24 agents, collective rule-breaking can be predicted from messages and that message content matters more than structure there. They do not establish how often real swarms tip or how monitors perform on real deployments. All shortcuts are file or merge manipulations inside a mock repository, and all agent code runs in a sandbox without network access. The injected messages are plain and were not optimised against any model, so the dataset does not provide a stronger attack than what is already public. The monitors read agent messages; deploying such monitors on human or mixed swarms would raise privacy questions that our work does not address. Two findings could help an adversary: the conditions that make swarms tip, and the blind spot for triggers that arrive through tool output. Both are ordinary properties of swarm setups, and we report them so that defenders can remove dead

ends and add checks outside the conversation. A monitor that becomes standard could also be gamed; we did not test agents that try to evade it.

Author Contributions

All three authors designed the study together, built the simulator, the monitors and the analysis, and wrote and reviewed this report.

References

Anagnostopoulos, A., Kumar, R., & Mahdian, M. (2008). Influence and correlation in social networks. Proc. KDD '08, 7–15. https://doi.org/10.1145/1401890.1401897

Cemri, M., et al. (2025). Why do multi-agent LLM systems fail? arXiv. https://arxiv.org/abs/2503.13657

Centola, D., & Macy, M. (2007). Complex contagions and the weakness of long ties. American Journal of Sociology, 113(3), 702–734. https://doi.org/10.1086/521848

Chan, C.-M., et al. (2024). AgentMonitor: A plug-and-play framework for predictive and secure multi-agent systems. arXiv. https://arxiv.org/abs/2408.14972

Flint Ashery, A., Aiello, L. M., & Baronchelli, A. (2025). Emergent social conventions and collective bias in LLM populations. Science Advances. https://doi.org/10.1126/sciadv.adu9368

Golechha, S., & Garriga-Alonso, A. (2025). Among Us: A sandbox for measuring and detecting agentic deception. NeurIPS 2025. https://arxiv.org/abs/2504.04072

Hammond, L., et al. (2025). Multi-agent risks from advanced AI. Cooperative AI Foundation, Technical Report #1. https://arxiv.org/abs/2502.14143

Hu, W., et al. (2020). Strategies for pre-training graph neural networks. ICLR 2020. https://arxiv.org/abs/1905.12265

Kutasov, J., et al. (2025). SHADE-Arena: Evaluating sabotage and monitoring in LLM agents. arXiv. https://arxiv.org/abs/2506.15740

Lee, D., & Tiwari, M. (2024). Prompt Infection: LLM-to-LLM prompt injection within multi-agent systems. arXiv. https://arxiv.org/abs/2410.07283

Luo, Z., et al. (2026). Auditing cascading risks in multi-agent systems via semantic-geometric co-evolution. arXiv. https://arxiv.org/abs/2603.13325

METR & Redwood Research (2026). Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident. https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/

Miao, R., et al. (2025). BlindGuard: Safeguarding LLM-based multi-agent systems under unknown attacks. arXiv. https://arxiv.org/abs/2508.08127

Mirakhor, K. (2026). Towards socially intelligent multi-agent systems: Zero-shot MARL coordination and theory-of-mind benchmarking of LLM agents for strategic deception. MSR thesis, Carnegie Mellon University, CMU-RI-TR-26-60.

Paglieri, D., et al. (2026). A case study on emergent cheating and whistleblowing in autonomous research swarms. arXiv. https://arxiv.org/abs/2609.04170

Wang, S., et al. (2025). G-Safeguard: A topology-guided security lens and treatment on LLM-based multi-agent systems. ACL 2025, 7261–7276. https://arxiv.org/abs/2502.11127

Yu, J., et al. (2026). Privacy-preserving topology-guided safety for LLM-based multi-agent systems via federated graph learning. arXiv. https://arxiv.org/abs/2609.02967

Yu, M., et al. (2024). NetSafe: Exploring the topological safety of multi-agent networks. arXiv. https://arxiv.org/abs/2410.15686

Zhao, Y. (2026). GRADE: Graph representation of LLM agent dependency and execution. arXiv. https://arxiv.org/abs/2606.22741

Zhong, Z., Raghunathan, A., & Carlini, N. (2025). ImpossibleBench: Measuring LLMs' propensity of exploiting test cases. arXiv. https://arxiv.org/abs/2510.20270

Zhou, J., et al. (2025). GUARDIAN: Safeguarding LLM multi-agent collaborations with temporal graph modeling. NeurIPS 2025. https://arxiv.org/abs/2505.19234

Appendix

A. Monitor details

Jury. Two judges (DeepSeek V4 Flash, Claude Haiku 5.5) rate every message once on nine dimensions from 0 to 3. Five are interpretive: rule break framed as legitimate, goal shift ("just make it green"), unverified authority, uncritical adoption, secrecy. Four are factual: success claim, workaround request, verification, frustration. Features use the judges' mean plus their disagreement. Messages are rated as data, never executed.

Graph. An edge i→j exists when agent j read a message from i in the last three rounds, either addressed to it (direct) or posted in a channel it reads (broadcast). The 18 edge features combine volume, responsiveness, reciprocity, new contacts, escalation words and the jury ratings (window maxima, means and trends). The 5 node features describe an agent's own broadcast posts, its stance drift, unanswered messages and role. No feature contains the round number, task keywords, tool events, model identity or configuration settings.

GNN. Relational GINE with separate weights for direct, broadcast and combined edges, a GRU memory per agent across rounds, a Poisson head that predicts the number of rule-breakers as an auxiliary target, and judge-swap augmentation (a training batch uses only one judge's ratings with probability 0.5). Five training runs, each averaging snapshots of its last eight epochs. The hybrid averages the GNN and boosting on the logit scale. Boosting is scikit-learn HistGradientBoosting (depth 3, 250 iterations).

LLM monitors. Claude Haiku 5.5 with thinking off returns a probability that the swarm tips within three rounds. The per-conversation and per-agent variants read a six-round window of one conversation or one agent and take the maximum over all of them; the whole-transcript variant reads rounds 1 to t and averages three samples. All seven variants on 252 runs cost $39 in total.

Evaluation. Cross-validation uses 18 folds on the training configurations: ten grouped folds and eight folds that each leave out one domain (shortcut type, model mix, channel layout, seed type, topology). Alarm thresholds are set at a 5% false-alarm rate on training data only.

B. Pre-registered hypotheses of the blind validation suite

	Prediction	Result	Verdict
H1	The GNN tells apart two runs of the same configuration	0.76 [0.45, 0.97] on 29 run pairs	Not supported
H2	The clock breaks down under a longer sprint	+0.08 [−0.09, +0.27] vs. replicate	Not supported
H3	The GNN is more stable than boosting under an unseen agent model	−0.01 [−0.19, +0.20]	Not supported
H4	The GNN keeps false alarms low under visible sanctions	AUROC stays level (+0.05); false alarms in 3 of 9 vs. 1 of 8 calm runs	Not supported
H5	The GNN gains under a hub network	+0.10 [−0.21, +0.61] vs. +0.01 for boosting	Supported (point rule)
H7	Replication on 27 fresh runs: the whole-transcript LLM beats the GNN	+0.03 [−0.05, +0.14]	Not supported

Table 3: Pre-registered predictions, scored mechanically by the rules fixed before each run. For H1–H5 no interval excludes its decision boundary; the suite resolves drops of about 0.2 AUROC. H7 was registered after H1–H5 were evaluated and before its 27 runs existed.